

Machine Learning — Comprehensive Study Guide

Topic 08 | University-Level Reference Document

Table of Contents

- 1. [Core Formula Reference](#)
- 2. [Chapter 1: Introduction to Data Science and Machine Learning](#)
- 3. [Chapter 2: Regression — Predicting Continuous Values](#)
- 4. [Chapter 3: Linear Regression — Deep Dive](#)
- 5. [Chapter 4: Classification](#)
- 6. [Chapter 5: Clustering — Unsupervised Learning](#)
- 7. [Chapter 6: K-Nearest Neighbors \(KNN\)](#)
- 8. [Chapter 7: Support Vector Machines \(SVM\)](#)
- 9. [Chapter 8: Other Important ML Algorithms](#)
- 10. [Chapter 9: Master Review](#)

Quick Reference: Core Formulas

Formula	Name
$Y = b_0 + b_1X$	Simple Linear Regression
$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$	Multiple Linear Regression
$b_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$	Slope
$b_0 = \frac{\sum y_i - b_1 \sum x_i}{n}$	Intercept
$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Mean Squared Error
$RMSE = \sqrt{MSE}$	Root Mean Squared Error
$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Mean Absolute Error
$R^2 = 1 - \frac{SSE}{SST}$	Coefficient of Determination
$d = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$	Euclidean Distance (n -D)

Formula	Name
$WCSS = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \ \mathbf{x} - \boldsymbol{\mu}_k\ ^2$	Within-Cluster Sum of Squares

Chapter 1: Introduction to Data Science and Machine Learning

1.1 What is Data Science?

Data Science is the interdisciplinary field of extracting useful knowledge and insights from data. It combines statistics, mathematics, programming, and domain expertise. The typical workflow involves collecting raw data, cleaning it, analysing it, and communicating findings to support informed decision-making.

1.2 What is Machine Learning?

Machine Learning (ML) is a core application of Data Science in which statistical, mathematical, and numerical techniques enable computers to learn from data automatically — without being explicitly programmed for every task.

When a model learns from data, it can:

- Summarise large datasets into concise conclusions
- Identify hidden patterns and visualise trends
- Group similar observations together
- Make predictions on new, unseen data

Important: Machine Learning is not magic. It is a collection of mathematical tools. It cannot replace careful human thinking and rigorous data preparation. Always question your data and validate your results.

1.3 Types of Machine Learning

Type	Definition	Key Feature	Example
Supervised Learning	The model learns from <i>labelled</i> data — data where the correct answer is already known	Labels are present	Predicting house prices from historical sales data
Unsupervised Learning	The model learns from <i>unlabelled</i> data — it finds hidden patterns without guidance	No labels	Grouping customers by purchasing behaviour

Decision rule: Ask yourself — *Does my data have labels (correct answers)?*

- **Yes** → Supervised Learning
- **No** → Unsupervised Learning

1.4 Overview of ML Algorithms

Algorithm	Type	Problem Solved
Linear Regression	Supervised	Predict a continuous value (e.g., price, height)
K-Nearest Neighbors (KNN)	Supervised	Regression or Classification
Classification Models	Supervised	Predict a category (e.g., spam / not spam)
K-Means Clustering	Unsupervised	Group data into K clusters
Support Vector Machine (SVM)	Supervised	Classification using a hyperplane
Neural Networks	Supervised	Complex pattern recognition
Random Forests	Supervised	Ensemble of decision trees
Gradient Boosted Trees	Supervised	Sequentially boosted ensemble of trees

1.5 Chapter Summary

- Data Science extracts knowledge from raw data using statistics, mathematics, and programming.
- Machine Learning is the automated application of statistical techniques to learn from data.
- **Supervised learning** requires labels; **unsupervised learning** does not.
- Regression predicts a number; Classification predicts a category; Clustering finds natural groups.

Chapter 2: Regression — Predicting Continuous Values

2.1 Definition

****Regression**** is a supervised learning task in which the goal is to predict a **continuous, real-valued** output. A continuous value can take any real number within a range — for example, 5.7, 100.3, or 99,999.

2.2 Real-World Applications

- Predicting a person's height given parental heights
- Estimating loan repayment time from credit history

- Forecasting package delivery time given traffic and weather
- Predicting a student's exam score from study hours
- Estimating a house's price from size, location, and room count

2.3 Regression vs. Classification

Aspect	Regression	Classification
Output type	Continuous number (e.g., 75.3)	Category or class (e.g., spam / not spam)
Example output	Predicted salary: \$45,200	Predicted label: Defective / Not Defective
Error measured by	MSE, RMSE, MAE	Accuracy, Precision, Recall
Simple example	Predict tomorrow's temperature	Predict if it will rain or not

2.4 Common Regression Techniques

- a) **Linear Regression** — Fits a straight line through data. The simplest and most widely used method (see Chapter 3).
- b) **K-Nearest Neighbors (KNN) Regression** — A non-parametric method. Finds the K training points closest to a new input (using Euclidean distance) and returns their average output value as the prediction.
- c) **Neural Networks** — Combines and transforms input variables through multiple non-linear layers. Powerful for complex, high-dimensional problems.
- d) **Random Forests** — Builds many decision trees on random subsets of data. The final prediction is the average across all trees.
- e) **Gradient Boosted Trees** — Builds trees sequentially; each new tree corrects the errors of the previous ensemble. Typically achieves higher accuracy than random forests.

2.5 Chapter Summary

- Regression predicts a continuous number; Classification predicts a category.
- KNN regression averages the K nearest neighbours.
- Random forests average many trees; gradient boosting corrects errors step by step.

Chapter 3: Linear Regression — Deep Dive

3.1 Definition

Linear Regression is a statistical method that models the relationship between one or more

input variables (features) and a continuous output variable (target) by fitting the best-possible straight line through the data.

3.2 Types of Linear Regression

Type	Independent Variables	Equation	When to Use
Simple Linear Regression	One (X)	$Y = b_0 + b_1X$	One feature predicts one output
Multiple Linear Regression	Many ($X_1, X_2, \dots, X_n$)	$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$	Many features predict one output

3.3 The Linear Regression Equation

For simple linear regression:

$$Y = b_0 + b_1X$$

Symbol	Name	Meaning
Y	Dependent Variable	The value to be predicted (output)
X	Independent Variable	The input feature (predictor)
b_0	Intercept	The value of Y when $X = 0$; where the line crosses the Y -axis
b_1	Slope (Regression Coefficient)	The change in Y for a one-unit increase in X

For multiple linear regression:

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Each coefficient b_j quantifies the change in Y when X_j increases by one unit, *holding all other features constant*.

✦ Diagram Placeholder: Regression Best-Fit Line

Description: A scatter plot of data points overlaid with the regression line. The line minimises the total squared vertical distances (residuals) from each point to the line. No single line passes through all points; the best-fit line minimises the aggregate error.

3.4 Ordinary Least Squares (OLS)

The goal of linear regression is to find b_0 and b_1 that best fit the data. This is achieved via **Ordinary Least Squares (OLS)**, which minimises the **Residual Sum of Squares (RSS)**:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Why square the differences?

- Squaring ensures all values are non-negative, preventing cancellation between positive and negative errors.
- Squaring penalises large errors more heavily than small ones.
- The resulting function is smooth and differentiable, making calculus-based optimisation tractable.

3.5 Formulas for Slope and Intercept

Slope:

$$b_1 = \frac{n \sum_{i=1}^n x_i y_i - (\sum_{i=1}^n x_i) (\sum_{i=1}^n y_i)}{n \sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2}$$

Intercept:

$$b_0 = \frac{\sum_{i=1}^n y_i - b_1 \sum_{i=1}^n x_i}{n} = \bar{y} - b_1 \bar{x}$$

Symbol	Meaning
n	Number of data points
x_i	The i -th input value
y_i	The i -th actual output value
$\sum x_i$	Sum of all X values
$\sum y_i$	Sum of all Y values
$\sum x_i y_i$	Sum of products of corresponding X and Y values
$\sum x_i^2$	Sum of squared X values

Given data:

X (Input)	Y (Output)
2	4
4	5
5	7
9	8

Step 1: Compute the required sums.

$$n = 4$$

$$\sum x_i = 2 + 4 + 5 + 9 = 20$$

$$\sum y_i = 4 + 5 + 7 + 8 = 24$$

$$\sum x_i^2 = 4 + 16 + 25 + 81 = 126$$

$$\sum x_i y_i = (2 \times 4) + (4 \times 5) + (5 \times 7) + (9 \times 8) = 8 + 20 + 35 + 72 = 135$$

Step 2: Calculate the slope b_1 .

$$b_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} = \frac{4(135) - (20)(24)}{4(126) - (20)^2} = \frac{540 - 480}{504 - 400} = \frac{60}{104} \approx 0.577$$

Step 3: Calculate the intercept b_0 .

$$b_0 = \frac{\sum y_i - b_1 \sum x_i}{n} = \frac{24 - (0.577)(20)}{4} = \frac{24 - 11.54}{4} = \frac{12.46}{4} \approx 3.115$$

Step 4: State the final regression equation.

$$\hat{Y} = 3.115 + 0.577 X$$

Interpretation:

- When $X = 0$, the predicted output is $\hat{Y} \approx 3.115$.
 - For each one-unit increase in X , $\hat{Y}$ increases by approximately 0.577.
-

3.7 Properties of the Regression Line

- The regression line always passes through the point $(\bar{x}, \bar{y})$.
- It minimises the sum of squared vertical distances from data points.
- b_0 is the Y -intercept; b_1 is the rate of change of Y with respect to X .

3.8 The Cost Function (MSE)

The **Mean Squared Error (MSE)** serves as the cost function in linear regression:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Symbol	Meaning
n	Number of data points
y_i	Actual value
$\hat{y}_i$	Predicted value
$y_i - \hat{y}_i$	Residual (prediction error)

The training objective is to **minimise MSE**, at which point the model achieves its best fit.

3.9 Model Performance Metrics — Full Worked Example

Dataset

House	Actual Price y_i	Predicted Price $\hat{y}_i$
1	100	110
2	150	140
3	200	180
4	250	240
5	300	290
6	350	320
7	400	380
8	450	420
9	500	480
10	550	520

The mean of actual prices:

$$\bar{y} = \frac{100 + 150 + \cdots + 550}{10} = 325$$

Full Computation Table

House	y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	$ y_i - \hat{y}_i $	$(y_i - \hat{y}_i)^2$	$\hat{y}_i - \bar{y}$	$(\hat{y}_i - \bar{y})^2$	$y_i - \bar{y}$	$(y_i - \bar{y})^2$
1	100	110	-10	10	100	-215	46 225	-225	50 625
2	150	140	10	10	100	-185	34 225	-175	30 625
3	200	180	20	20	400	-145	21 025	-125	15 625
4	250	240	10	10	100	-85	7 225	-75	5 625
5	300	290	10	10	100	-35	1 225	-25	625
6	350	320	30	30	900	-5	25	25	625
7	400	380	20	20	400	55	3 025	75	5 625
8	450	420	30	30	900	95	9 025	125	15 625
9	500	480	20	20	400	155	24 025	175	30 625
10	550	520	30	30	900	195	38 025	225	50 625
Total				190	4 300		184 025		206 250

Metric 1 — Sum of Squares Regression (SSR)

$$\text{SSR} = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 \quad \uparrow$$

****Interpretation:**** How much of the total variation in Y is *explained* by the model. A higher SSR indicates a better-fitting model.

Calculation:

$$\text{SSR} = 46,225 + 34,225 + 21,025 + 7,225 + 1,225 + 25 + 3,025 + 9,025 + 24,025 + 38,025 = 184,025$$

Metric 2 — Sum of Squares Error (SSE)

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Interpretation: How much variation remains *unexplained* by the model. A lower SSE indicates smaller prediction errors.

Calculation:

$$\text{SSE} = 100 + 100 + 400 + 100 + 100 + 900 + 400 + 900 + 400 + 900 = 4,300$$

Metric 3 — Sum of Squares Total (SST)

$$\text{SST} = \sum_{i=1}^n (y_i - \bar{y})^2$$

Interpretation: The total variance present in the actual output values, regardless of any model.

Calculation:

$$\text{SST} = 50,625 + 30,625 + 15,625 + 5,625 + 625 + 625 + 5,625 + 15,625 + 30,625 + 50,625 = 206,250$$

Important identity (holds exactly only for OLS-fitted models):

$$\text{SST} = \text{SSR} + \text{SSE}$$

Note: Because the predicted values in this example are not produced by an OLS regression on this specific dataset, the identity $\text{SST} = \text{SSR} + \text{SSE}$ does not hold exactly here ($184,025 + 4,300 = 188,325 \neq 206,250$). For any properly fitted OLS model, this decomposition holds precisely. This is an important concept to understand for examinations.

Metric 4 — Mean Squared Error (MSE)

$$\text{MSE} = \frac{\text{SSE}}{n} = \frac{4,300}{10} = 430$$

Interpretation: The average squared prediction error. Smaller is better. Because errors are squared, large mistakes are penalised disproportionately.

Metric 5 — Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{430} \approx 20.74$$

Interpretation: On average, predictions deviate from actual values by approximately 20.74 units. RMSE is expressed in the same units as the target variable (e.g., dollars, degrees, kilograms), making it more interpretable than MSE.

Metric 6 — Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| = \frac{10 + 10 + 20 + 10 + 10 + 30 + 20 + 30 + 20 + 30}{10} = \frac{190}{10} = 19$$

Interpretation: On average, predictions are off by 19 units. MAE treats all errors equally (no squaring), making it more robust to outliers than MSE or RMSE.

Metric 7 — R² Score (Coefficient of Determination)

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{4,300}{206,250} = 1 - 0.0208 \approx 0.9792$$

Interpretation: The model explains approximately **97.92%** of the total variation in housing prices — a very strong result.

<i>R</i> ² Value	Meaning
0.00	Model explains nothing
0.30	Weak model
0.50	Moderate model
0.70	Good model
0.90+	Excellent model

Summary: All Metrics

Metric	Formula	Result
SSR	$\sum(\hat{y}_i - \bar{y})^2$	184 025
SSE	$\sum(y_i - \hat{y}_i)^2$	4 300
SST	$\sum(y_i - \bar{y})^2$	206 250
MSE	SSE/n	430
RMSE	$\sqrt{MSE}$	20.74
MAE	$\frac{1}{n} \sum y_i - \hat{y}_i $	19
R^2	$1 - SSE/SST$	0.9792

3.10 Choosing the Right Error Metric

Metric	Sensitive to Outliers?	Units	Best Used When
MAE	No (absolute values)	Same as Y	Outliers should not dominate the measure
MSE	Yes (squares large errors)	Y^2	Large errors must be penalised heavily
RMSE	Yes	Same as Y	Penalty for large errors AND interpretable scale needed

3.11 Advantages and Limitations

Advantages	Limitations
Simple and interpretable	Assumes a linear relationship between X and Y
Fast to train	Sensitive to outliers
Coefficients explain model behaviour	Cannot model complex non-linear patterns
Reliable when the relationship is truly linear	Requires features to be independent (no multicollinearity)
Good baseline for comparison	Assumes constant variance of errors (homoscedasticity)

- Linear regression fits $\hat{Y} = b_0 + b_1 X$ (simple) or $\hat{Y} = b_0 + \sum b_j X_j$ (multiple).
- b_0 is the Y -intercept; b_1 is the slope (change in $\hat{Y}$ per unit of X).
- OLS minimises RSS to determine b_0 and b_1 .
- MSE, RMSE, MAE quantify prediction error; R^2 quantifies explanatory power.
- The identity $SST = SSR + SSE$ holds exactly only for OLS-fitted models.

Chapter 4: Classification

4.1 Definition

Classification is a supervised learning task in which the model predicts the *category* (class) that a data point belongs to. Unlike regression, which outputs a number, classification outputs a label.

Examples:

- Is this email spam or not spam? (*Binary*)
- What object is in this image: cat, dog, or bird? (*Multiclass*)
- Is this manufactured part defective? (*Binary*)
- Which handwritten digit (0–9) is shown? (*Multiclass*)

4.2 Types of Classification

Type	Number of Classes	Example
Binary Classification	2	Spam / Not Spam; Positive / Negative
Multiclass Classification	3 or more	Classify images as: cat, dog, or bird

4.3 Lazy vs. Eager Learners

✦ **Diagram Placeholder: Lazy vs. Eager Learner Comparison**

Description: Two-column diagram contrasting the training and prediction phases of lazy learners (e.g., KNN) and eager learners (e.g., Decision Trees, Neural Networks).

Aspect	Lazy Learner (e.g., KNN)	Eager Learner (e.g., Decision Tree, Neural Network)
Training time	Very short — only stores data	Long — builds a complete model
Prediction time	Slow — computes distance to all training points	Fast — applies the pre-built model
Memory use	Stores all training data	Stores only the learned model parameters
Example	K-Nearest Neighbors	Linear Regression, Random Forest, SVM

4.4 Chapter Summary

- Classification predicts a label; regression predicts a number.
- Binary = 2 classes; Multiclass = 3 or more classes.
- Classification is always supervised (labelled training data required).
- Lazy learners train fast but predict slowly; eager learners train slowly but predict fast.

Chapter 5: Clustering — Unsupervised Learning

5.1 Definition

Clustering is an unsupervised learning task in which data points are grouped into *clusters* such that points within the same cluster are similar to each other, and points in different clusters are dissimilar. No labels are used; the algorithm discovers natural groupings on its own.

5.2 Applications

- Discovering hidden patterns without prior knowledge
- Segmenting customers by purchasing behaviour for targeted marketing
- Identifying topics or document types in text corpora
- Detecting anomalies as outlier clusters
- Compressing data by representing each cluster with its centroid

5.3 Hard vs. Soft Clustering

Aspect	Hard Clustering	Soft Clustering
Assignment	Each point belongs to exactly one cluster	Each point belongs to all clusters with varying probabilities

Aspect	Hard Clustering	Soft Clustering
Output	A single cluster label per point	A probability distribution over clusters per point
Example	K-Means	Fuzzy C-Means, Gaussian Mixture Models
Use case	When cluster boundaries are clear	When boundaries are ambiguous

5.4 Types of Clustering Methods

a) Partitioning (Centroid-based) — Divides data into K non-overlapping groups. Each point is assigned to the cluster with the nearest centroid. Example: **K-Means**.

b) Density-based — Clusters are dense regions of data surrounded by sparser regions. Effective for non-spherical cluster shapes. Example: **DBSCAN**.

****c) Hierarchical**** — Builds a tree of clusters (a **dendrogram**). Does not require K to be specified in advance.

5.5 K-Means Clustering — Deep Dive

Definition

K-Means partitions data into K clusters by iteratively assigning each point to its nearest centroid and updating centroids to be the mean of their assigned points.

K-Means is a classic **Expectation-Maximisation (EM)** algorithm:

- **E-step:** Assign each point to the nearest centroid.
- **M-step:** Recompute each centroid as the mean of its assigned points.

Key Goal: Maximise *intra-cluster* similarity (points within a cluster are close together) and minimise *inter-cluster* similarity (points in different clusters are far apart).

Algorithm — Step by Step

1. Choose K (the number of clusters).
2. Randomly select K initial seed points as the starting centroids.
3. Compute the squared Euclidean distance from each data point to each centroid.
4. Assign each point to the cluster with the closest centroid.
5. Recompute each centroid as the mean of all points assigned to it.
6. Repeat steps 3-5 until cluster assignments no longer change, or a maximum number of iterations is reached.

Description: A sequence of scatter plots showing: (1) random centroid initialisation, (2) initial assignment of points, (3) centroid recalculation, (4) re-assignment, and (5) final converged clusters.

Distance Measure in K-Means

K-Means uses **Squared Euclidean Distance** for efficiency:

$$d^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

Practical note: All feature dimensions should have similar scales. If one feature is measured in thousands and another in single digits, the large-scale feature will dominate the distance calculation. Rescale features (e.g., standardisation or min-max scaling) before applying K-Means.

5.6 WCSS — Within-Cluster Sum of Squares

$$\text{WCSS} = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2$$

Where K is the number of clusters, C_k is the set of points in cluster k , and $\boldsymbol{\mu}_k$ is the centroid of cluster k .

K-Means minimises WCSS. Lower WCSS indicates more compact, tighter clusters.

5.7 Choosing K — The Elbow Method

The primary challenge in K-Means is selecting an appropriate K . The **Elbow Method** provides a systematic approach:

1. Run K-Means for $K = 1, 2, 3, \dots, 10$.
 2. For each K , record WCSS.
 3. Plot WCSS against K .
 4. Identify the *elbow* — the point beyond which WCSS decreases only marginally.
 5. Choose the K at the elbow as the optimal number of clusters.
-

✦ Diagram Placeholder: Elbow Curve

**Description:* A line chart with K on the x -axis and WCSS on the y -axis. The curve decreases steeply for small K and flattens after the optimal K , forming a characteristic

| elbow shape.

5.8 K-Means Strengths and Weaknesses

Strengths	Weaknesses
Simple and easy to implement	Must specify K in advance
Scales well to large datasets	Sensitive to initial centroid placement (may converge to a local optimum)
Fast convergence in practice	Assumes spherical, similarly sized clusters
Easy to interpret	Not designed for non-numeric (categorical) data
Good baseline clustering method	No guarantee of finding the globally optimal solution

5.9 Practical Tips

- Rescale features before running K-Means.
- Run K-Means multiple times with different random initialisations; retain the result with the lowest WCSS.
- Use the Elbow Method to guide the choice of K .

5.10 Chapter Summary

- Clustering is unsupervised grouping of similar data points.
 - Hard clustering assigns each point to exactly one cluster; soft clustering uses probabilities.
 - K-Means assigns points to the nearest centroid, recalculates centroids, and repeats until convergence.
 - WCSS measures cluster compactness — minimise it.
 - Use the Elbow Method to select the optimal K .
-

Chapter 6: K-Nearest Neighbors (KNN)

6.1 Definition

K-Nearest Neighbors (KNN) is a simple, non-parametric, supervised learning algorithm applicable to both regression and classification. Its core principle is that a data point is likely to share properties with its nearest neighbours in feature space.

6.2 KNN for Classification

1. Store all labelled training data.
2. For a new (unlabelled) data point, compute its distance to every training point.
3. Identify the K nearest training points.
4. Take a **majority vote**: the class most frequent among the K neighbours is the prediction.

Intuition: Imagine moving to a new city and judging whether a neighbourhood is safe by consulting the K nearest residents. If 4 out of 5 say it is safe, you conclude it is safe.

6.3 KNN for Regression

1. Store all labelled training data.
2. For a new data point, compute its distance to every training point.
3. Identify the K nearest training points.
4. **Average** their target values — that average is the prediction.

6.4 Distance Calculation

The standard distance measure in KNN is **Euclidean Distance**:

In 2D:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In n dimensions (general form):

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

✦ Diagram Placeholder: KNN Classification Example

***Description:** A scatter plot with two labelled classes (e.g., A and B). A new unlabelled point is shown, with circles drawn around its K nearest neighbours. The majority class among those neighbours determines the prediction.

6.5 Choosing K

K Value	Behaviour	Risk
Small (e.g., $K = 1$)	Very sensitive to individual data points	Overfitting, high variance
Large (e.g., $K = 100$)	Smooth, generalised decision boundary	Underfitting, high bias

- K is typically chosen to be **odd** to avoid ties in binary classification.
- Standard practice: test multiple values of K and select the one with the best validation performance.

6.6 KNN Properties

Property	Detail
Training time	Negligible — only stores data
Prediction time	Slow — computes distances to all training points per query
Memory	High — all training data must be retained
Parametric?	No — non-parametric; no assumptions about data distribution
Applicable to	Both regression and classification

6.7 K-Means vs. KNN — Key Distinction

Aspect	K-Means	KNN
Learning type	Unsupervised	Supervised
Meaning of K	Number of clusters to create	Number of nearest neighbours to consult
Goal	Group unlabelled data into K clusters	Classify or predict a value for a new point
Training	Iterative centroid updates	Lazy: stores all training data
Requires labels?	No	Yes
Output	Cluster assignment	Class label or predicted value

6.8 Advantages and Limitations

Advantages	Limitations
Extremely simple to understand and implement	Slow prediction — computes distances to all training points
No training phase required	High memory usage — stores entire training set
Naturally handles multiclass problems	Sensitive to irrelevant features and feature scale
Non-parametric — no distributional assumptions	Degrades in high-dimensional spaces (curse of dimensionality)

6.9 Chapter Summary

- KNN is a lazy, non-parametric supervised algorithm.
- Classification: majority vote among K neighbours; Regression: average of K neighbours.
- Uses Euclidean distance to identify nearest neighbours.
- Small $K \rightarrow$ high variance (overfitting risk); Large $K \rightarrow$ high bias (underfitting risk).

Chapter 7: Support Vector Machines (SVM)

7.1 Definition

Support Vector Machine (SVM) is a supervised classification algorithm that finds the **optimal hyperplane** — the decision boundary that maximally separates two classes in feature space.

7.2 What is a Hyperplane?

A hyperplane is a flat subspace of dimension one less than the feature space:

- In **2D**: a line ($w_1x_1 + w_2x_2 + b = 0$)
- In **3D**: a plane
- In n **D**: a hyperplane

✦ Diagram Placeholder: SVM Hyperplane and Support Vectors

Description: A 2D scatter plot with two classes (e.g., circles and crosses). The optimal hyperplane (line) is drawn between them. The margin is shown as the region between two dashed parallel lines. The support vectors — the data points lying exactly on the margin boundaries — are highlighted.

7.3 Key SVM Concepts

Concept	Definition
Hyperplane	The decision boundary separating two classes. SVM selects the optimal hyperplane.
Support Vectors	The training points closest to the hyperplane. They lie on the margin boundaries and define the hyperplane's position.
Margin	The perpendicular distance between the hyperplane and the nearest support vectors on each side.
Maximum Margin Classifier	The hyperplane that achieves the largest possible margin between the two classes.

7.4 Why Maximise the Margin?

A wider margin implies greater confidence in predictions. It also improves **generalisation** — models with a larger margin tend to perform better on new, unseen data because the decision boundary is further from all training points.

7.5 Strengths and Limitations of SVM

Strengths	Limitations
Effective in high-dimensional spaces	Training can be slow on large datasets
Memory-efficient — uses only support vectors	Sensitive to feature scaling
Versatile via kernel functions (linear, RBF, polynomial)	Choice of kernel and hyperparameters requires careful tuning
Robust to overfitting in high dimensions	Not directly interpretable like linear regression

7.6 Chapter Summary

- SVM finds the hyperplane with the maximum margin between two classes.
- Support vectors are the training points closest to the hyperplane; they define the margin.
- In 2D the hyperplane is a line; in n D it is a $(n-1)$ -dimensional subspace.
- Maximising the margin improves generalisation to unseen data.

Chapter 8: Other Important ML Algorithms

8.1 Neural Networks

A **neural network** is a supervised learning model loosely inspired by the human brain. It processes inputs through layers of connected "neurons."

How it works:

1. Receive input features (numerical vectors).
2. At each neuron: compute a weighted sum of inputs and add a bias term.
3. Apply a non-linear **activation function** (e.g., ReLU, sigmoid).
4. Pass the output to the next layer.
5. Repeat through many layers until the final output is produced.

The key power of neural networks lies in **non-linearity** — applying non-linear functions at each layer allows the network to learn arbitrarily complex patterns that linear models cannot capture.

8.2 Random Forests

A **random forest** is an ensemble method that builds many decision trees during training and aggregates their predictions.

- Each tree is trained on a **random subset** of the data (bootstrap sampling).
- Each tree uses a **random subset of features** at each split.
- Final prediction: **average** (regression) or **majority vote** (classification).

Why randomness? Diversity among trees prevents them from all making the same errors. Different trees capture different patterns; together they cancel out individual errors — this is called **ensemble learning**.

8.3 Gradient Boosted Trees

Unlike random forests (which build trees in parallel), **gradient boosted trees** build trees *sequentially*:

1. Start with a simple prediction (e.g., $\hat{Y} = \bar{y}$).
2. Compute the residuals (errors) from the current model.
3. Build a new tree that predicts these residuals.
4. Add the new tree to the ensemble with a small learning rate.
5. Repeat until performance converges or a maximum number of iterations is reached.

8.4 Algorithm Comparison

Algorithm	Type	Training	Prediction	Best For	Weakness
KNN	Supervised	Fast (lazy)	Slow	Small datasets,	Slow prediction,

Algorithm	Type	Training	Prediction	Best For	Weakness
				simple boundaries	high memory
Linear Regression	Supervised	Fast	Very fast	Linear relationships	Cannot model non-linearity
K-Means	Unsupervised	Iterative	Fast	Simple clustering	Must specify K ; local optima
SVM	Supervised	Slow (large data)	Fast	High-dimensional data, clear margins	Slow training, kernel tuning
Neural Networks	Supervised	Slow	Fast	Complex non-linear patterns	Requires large data; less interpretable
Random Forest	Supervised	Moderate	Moderate	Tabular data, robust predictions	Less interpretable than a single tree
Gradient Boosting	Supervised	Slow	Fast	Competition benchmarks, tabular data	Can overfit; slow to train

Chapter 9: Master Review

9.1 All Key Formulas

Formula	Name	Purpose
$\hat{Y} = b_0 + b_1X$	Simple Linear Regression	Predict Y from one input X
$\hat{Y} = b_0 + b_1X_1 + \dots + b_nX_n$	Multiple Linear Regression	Predict Y from many inputs
$b_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$	Slope	Compute regression slope
$b_0 = \frac{\sum y_i - b_1 \sum x_i}{n}$	Intercept	Compute Y -intercept
$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Mean Squared Error	Average squared error
$RMSE = \sqrt{MSE}$	Root Mean Squared Error	Error in original units
$MAE = \frac{1}{n} \sum y_i - \hat{y}_i $	Mean Absolute Error	Outlier-robust average error

Formula	Name	Purpose
$R^2 = 1 - \frac{SSE}{SST}$	R-Squared	Proportion of variance explained
$SST = SSR + SSE$	Variance Decomposition	Total = Explained + Unexplained (OLS)
$d = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$	Euclidean Distance	Used in KNN and K-Means
$WCSS = \sum_k \sum_{\mathbf{x} \in C_k} \ \mathbf{x} - \boldsymbol{\mu}_k\ ^2$	WCSS	Cluster compactness (K-Means)

9.2 All Key Definitions

Term	Definition
Machine Learning	Teaching computers to learn from data using statistical techniques
Supervised Learning	Learning from labelled data where correct answers are known
Unsupervised Learning	Finding patterns in data without any labels
Regression	Predicting a continuous numerical output
Classification	Predicting a discrete category or class
Clustering	Unsupervised grouping of data by similarity
Hyperplane	The decision boundary in SVM that separates classes
Support Vector	The training point closest to the SVM hyperplane; defines the margin
Margin	Distance between the hyperplane and the nearest support vectors
Centroid	The mean position of all points in a K-Means cluster
WCSS	Sum of squared distances from each point to its cluster centroid
KNN	Lazy supervised learner; predicts using K nearest training neighbours
Intercept (b_0)	Value of $\hat{Y}$ when $X = 0$; where the regression line crosses the Y -axis
Slope (b_1)	Change in $\hat{Y}$ per unit increase in X
R^2	Proportion of variance in Y explained by the model; ranges from 0 to 1
Overfitting	Model fits training data well but generalises poorly to new data
Cost Function	Quantifies prediction error; the model minimises this during training

Term	Definition
Ensemble Method	Combining many models for improved performance (e.g., random forest)

9.3 Common Exam Questions

Q1: What is the difference between regression and classification? Regression predicts a continuous number (e.g., house price = \$250,000). Classification predicts a discrete category (e.g., email = spam). Both are supervised learning tasks.

Q2: What does K mean in KNN vs. K-Means? In KNN, K is the number of nearest neighbours consulted to make a prediction. In K-Means, K is the number of clusters into which data is partitioned. The letter K is coincidental — the two algorithms and their objectives are entirely different.

Q3: How does K-Means find clusters? K-Means randomly initialises K centroids, assigns each point to the nearest centroid, recomputes each centroid as the mean of its assigned points, and repeats until centroids no longer move.

Q4: What is R^2 and what does it mean? R^2 measures the proportion of total variance in Y explained by the model. $R^2 = 0$ means the model explains nothing; $R^2 = 1$ means the model explains all variation perfectly.

Q5: What is a support vector in SVM? A support vector is a training point lying exactly on the margin boundary — one of the points closest to the decision hyperplane. These critical points define the hyperplane's position and the width of the margin.

9.4 Practice Problems

- Given $X = [1, 2, 3, 4, 5]$ and $Y = [2, 4, 5, 4, 5]$, compute the linear regression equation $\hat{Y} = b_0 + b_1X$ using the OLS formulas.
- A K-Means algorithm starts with $K = 2$, initial centroids at 100 and 500. Data = $\{80, 120, 450, 530\}$. Perform one full iteration: assign each point to the nearest centroid and recompute the centroids.
- Using KNN with $K = 3$, predict the class of the point $(3, 3)$ given training data: $(1, 1) \rightarrow A$, $(2, 2) \rightarrow A$, $(4, 4) \rightarrow B$, $(5, 5) \rightarrow B$, $(3, 5) \rightarrow B$.
- If $SST = 1,000$ and $SSE = 400$, compute R^2 . What does this value mean in practical terms?
- Explain why WCSS always decreases (or stays constant) as K increases in K-Means, and why this does not mean that a larger K is always better.

9.5 Quick Reference Card

Topic	Key Points
Linear Regression	$\hat{Y} = b_0 + b_1 X$. Minimise MSE. Evaluate with R^2 , RMSE, MAE.
Classification	Predict a category. Supervised. Lazy (KNN) vs. eager (trees, SVM) learners.
K-Means	Unsupervised. Assign to nearest centroid $\rightarrow$ recalculate $\rightarrow$ repeat. Minimise WCSS. Elbow method selects K .
KNN	Supervised. Lazy. Find K nearest neighbours $\rightarrow$ vote (classification) or average (regression). Uses Euclidean distance.
SVM	Find the hyperplane with maximum margin. Support vectors define the margin. Larger margin $\rightarrow$ better generalisation.
Random Forest	Ensemble of random trees. Average (regression) or majority vote (classification). Reduces overfitting through diversity.
Gradient Boosting	Sequential trees. Each tree corrects residuals of the previous model. High accuracy; prone to overfitting without regularisation.

Appendix A: Distance Formulas — Complete Reference

Distance metrics underpin many ML algorithms. KNN uses distance to find neighbours; K-Means uses it to assign points to centroids; SVMs implicitly use it in the kernel geometry.

A.1 Euclidean Distance (2D)

The straight-line distance between two points $A = (x_1, y_1)$ and $B = (x_2, y_2)$:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Example: $A = (6, 6), B = (-6, -6)$

$$d = \sqrt{(6 - (-6))^2 + (6 - (-6))^2} = \sqrt{144 + 144} = \sqrt{288} \approx 16.97$$

✦ Diagram Placeholder: Euclidean Distance (2D)

***Description:** A coordinate plane showing points $A(6, 6)$ and $B(-6, -6)$. A line segment connects them, labelled $d \approx 16.97$. Dashed lines indicate $\Delta x = 12$ and $\Delta y = 12$, forming a right triangle.

A.2 Euclidean Distance (n Dimensions)

For two points $\mathbf{a} = (a_1, a_2, \dots, a_n)$ and $\mathbf{b} = (b_1, b_2, \dots, b_n)$ in n -dimensional space:

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

This generalises the 2D formula to any number of features.

A.3 Squared Euclidean Distance

$$d^2(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n (a_i - b_i)^2$$

The square root is omitted, which is computationally faster. It preserves the **ordering** of distances (if $d_1 < d_2$ then $d_1^2 < d_2^2$), so the nearest neighbour is identical under either metric.

Used in:

- K-Means (centroid assignment)
- KNN (when only rank order of distances matters)
- Optimisation problems where the square root is unnecessary overhead

A.4 Distance Metric Comparison

Metric	Formula	Sensitive to Scale?	Used In
Euclidean	$\sqrt{\sum (a_i - b_i)^2}$	Yes	KNN, K-Means
Squared Euclidean	$\sum (a_i - b_i)^2$	Yes	K-Means (faster), OLS
Manhattan	$\sum a_i - b_i $	Yes	Sparse data, grid problems
Cosine Similarity	$\frac{\mathbf{a} \cdot \mathbf{b}}{\ \mathbf{a}\ \ \mathbf{b}\ }$	No (direction only)	Text/NLP similarity

Practical note: All Euclidean-based metrics are sensitive to feature scale. Always standardise (zero mean, unit variance) or normalise (min-max scale) features before computing distances.

Appendix B: Notation Glossary

Consistent notation is used throughout this document. The table below is a definitive reference.

Symbol	Meaning
x_i	The i -th input (feature) value
y_i	The i -th actual (observed) output value
$\hat{y}_i$	The i -th predicted output value
$\bar{y}$	Mean of all actual output values: $\bar{y} = \frac{1}{n} \sum y_i$
$\bar{x}$	Mean of all input values
n	Number of data points (sample size)
b_0	Regression intercept
b_1	Regression slope (simple linear regression)
b_j	Regression coefficient for feature X_j (multiple regression)
K	Number of clusters (K-Means) or neighbours (KNN) — context-dependent
C_k	The set of data points assigned to cluster k
μ_k	Centroid (mean vector) of cluster k
$\ \cdot\ $	Euclidean norm: $\ \mathbf{v}\ = \sqrt{\sum v_i^2}$
SST	Sum of Squares Total: $\sum (y_i - \bar{y})^2$
SSR	Sum of Squares Regression: $\sum (\hat{y}_i - \bar{y})^2$
SSE	Sum of Squares Error: $\sum (y_i - \hat{y}_i)^2$
MSE	Mean Squared Error: SSE/n
RMSE	Root Mean Squared Error: $\sqrt{\text{MSE}}$
MAE	Mean Absolute Error: $\frac{1}{n} \sum y_i - \hat{y}_i $
R^2	Coefficient of Determination: $1 - \text{SSE}/\text{SST}$
WCSS	Within-Cluster Sum of Squares: $\sum_k \sum_{\mathbf{x} \in C_k} \ \mathbf{x} - \mu_k\ ^2$

Appendix C: Practice Problem Solutions

C.1 Linear Regression — Practice Problem 1

Given: $X = [1, 2, 3, 4, 5], Y = [2, 4, 5, 4, 5]$

Step 1: Compute sums.

$$n = 5, \quad \sum x_i = 15, \quad \sum y_i = 20$$

$$\sum x_i^2 = 1 + 4 + 9 + 16 + 25 = 55$$

$$\sum x_i y_i = (1)(2) + (2)(4) + (3)(5) + (4)(4) + (5)(5) = 2 + 8 + 15 + 16 + 25 = 66$$

Step 2: Compute slope.

$$b_1 = \frac{5(66) - (15)(20)}{5(55) - (15)^2} = \frac{330 - 300}{275 - 225} = \frac{30}{50} = 0.6$$

Step 3: Compute intercept.

$$b_0 = \frac{20 - (0.6)(15)}{5} = \frac{20 - 9}{5} = \frac{11}{5} = 2.2$$

Result:

$\hat{Y} = 2.2 + 0.6 X$

C.2 K-Means — Practice Problem 2

Given: Data = {80, 120, 450, 530}, initial centroids $\mu_1 = 100, \mu_2 = 500$.

Step 1: Assign each point to the nearest centroid.

Point	$d \text{ to } \mu_1 = 100$	$d \text{ to } \mu_2 = 500$	Assigned Cluster
80	20	420	Cluster 1
120	20	380	Cluster 1
450	350	50	Cluster 2
530	430	30	Cluster 2

Step 2: Recompute centroids.

$$\mu_1^{\text{new}} = \frac{80 + 120}{2} = 100 \quad \mu_2^{\text{new}} = \frac{450 + 530}{2} = 490$$

After one iteration: $\mu_1 = 100, \mu_2 = 490$. Assignments are unchanged, so the algorithm has converged.

C.3 KNN — Practice Problem 3

Given: Training data: $(1, 1) \rightarrow A, (2, 2) \rightarrow A, (4, 4) \rightarrow B, (5, 5) \rightarrow B, (3, 5) \rightarrow B$. Predict class of $(3, 3)$ with $K = 3$.

Step 1: Compute Euclidean distances from $(3, 3)$ to each training point.

Point	Class	Distance
$(1, 1)$	A	$\sqrt{(3-1)^2 + (3-1)^2} = \sqrt{8} \approx 2.83$
$(2, 2)$	A	$\sqrt{(3-2)^2 + (3-2)^2} = \sqrt{2} \approx 1.41$
$(4, 4)$	B	$\sqrt{(3-4)^2 + (3-4)^2} = \sqrt{2} \approx 1.41$
$(5, 5)$	B	$\sqrt{(3-5)^2 + (3-5)^2} = \sqrt{8} \approx 2.83$
$(3, 5)$	B	$\sqrt{(3-3)^2 + (3-5)^2} = \sqrt{4} = 2.00$

Step 2: Select the 3 nearest neighbours.

Rank	Point	Class	Distance
1 (tie)	$(2, 2)$	A	1.41
1 (tie)	$(4, 4)$	B	1.41
3	$(3, 5)$	B	2.00

Step 3: Majority vote among $K = 3$ neighbours.

- Class A: 1 vote
- Class B: 2 votes

Predicted class: B

C.4 R^2 — Practice Problem 4

Given: $SST = 1,000$, $SSE = 400$.

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{400}{1,000} = 1 - 0.4 = \boxed{0.6}$$

Interpretation: The model explains 60% of the total variation in Y . The remaining 40% (SSE/SST) is unexplained error. This is a moderate model; improvements to feature engineering or model complexity may increase R^2 further.

End of Study Guide — Topic 08